VOLUME III · THE GAPS · BINDING, SCOPING, PARADIGMS AND TRANSLATION

CMS 710 — Beyond the Four Modules

The class modules cover language definition, types, data structures and control flow. A Principles of Programming Languages paper also draws on binding times, static versus dynamic scoping, the full family of parameter-passing models, language paradigms and how translation actually works. This volume is that material, aligned to what the modules already say.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Wednesday 12 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturers:** the lecturers

READ THIS IN THE RIGHT ORDER

§1 and §2 are direct extensions of the lecturer's own test questions — he asked about scope and lifetime, and the natural follow-up is **binding times** and **static versus dynamic scoping**; he asked about pass by value and reference, and the natural follow-up is **the other three passing models**. Learn those two sections first. **§3–§6** are standard PPL topics the modules never reach; take them after Volumes I and II are secure.

1 Names, bindings and the six attributes of a variable

Module 4 says a variable has a **scope** and a **lifetime**. In fact a variable is best understood as **a six-tuple of attributes**, and scope and lifetime are two of the six. This framing turns a two-item answer into a six-item one.

#	ATTRIBUTE	WHAT IT IS
1	**Name**	The **identifier** used to refer to it in the source text
2	**Address**	The **memory location** it occupies — its **l-value**
3	**Type**	The **set of values and permitted operations** — Module 2's definition
4	**Value**	The **contents** of that location — its **r-value**
5	**Lifetime**	The **period during which it exists in memory** — Module 4's definition
6	**Scope**	**Where in the program it is visible** — Module 4's definition

l-value and r-value — worth one line. In `x = y`, the **l-value** of `x` is used (its address, where the result goes) and the **r-value** of `y` is used (its contents). This is why `10 = x` is illegal: a literal has an r-value but no l-value.

ALIASES

Two names bound to the **same memory address** are **aliases**. Pass-by-reference creates one deliberately: inside `void f(int &x)`, the name `x` and the caller's variable are aliases for one location. Aliasing is powerful but **damages readability and reliability**, because a change through one name silently changes the other — which is precisely the risk column in the pass-by-reference table.

BINDING AND BINDING TIME EXTENDS THE SCOPE QUESTION

A **binding** is an **association between an attribute and an entity** — between a name and a type, or a name and a memory address. **Binding time** is **when that association is made**.

BINDING TIME	EXAMPLE OF WHAT IS BOUND THEN
Language design time	The meaning of ∗ as multiplication; the set of keywords
Language implementation time	The range of `int`; how floating point is represented
Compile time	A variable's **type** in C++ — `int x;`
Link time	A call to a library function bound to its actual code
Load time	A **global variable bound to its address** when the program loads
Run time	A **local variable bound to a stack address** when its function is entered; a variable's **value** whenever it is assigned

KIND	DEFINITION	EXAMPLE
Static binding	Occurs **before run time** and **does not change** during execution	C++ `int x = 10;` — the type is fixed at compile time
Dynamic binding	Occurs **during execution** and **may change**	Python `x = 10` then `x = "CMS710"` — the type is rebound

This unifies Module 2 and Module 4. Static typing is early type binding; dynamic typing is late type binding. And the lifetime distinction in the lecturer's Q3 is a **storage binding** question: a global's address is bound at **load time** and never released, a local's at **run time** on entry and released on return. Saying it that way shows the two questions are one idea.

Storage bindings — the four categories

CATEGORY	WHEN STORAGE IS ALLOCATED	EXAMPLE	ADVANTAGE / DISADVANTAGE
Static	**Before execution begins**, and kept for the whole program	C++ `static` variables · globals	Efficient, direct addressing, values survive calls · but no memory reuse and no recursion support
Stack-dynamic	**When the declaration is elaborated**, i.e. when the block is entered	Ordinary local variables	**Allows recursion** and shares memory between calls · but costs allocation time and indirect addressing
Explicit heap-dynamic	By an **explicit instruction** from the programmer at run time	C++ `new` / `delete`	Full flexibility · but unreliable and costly — **leaks and dangling pointers**
Implicit heap-dynamic	**Automatically, on assignment**	Python lists and objects; JavaScript arrays	Maximum flexibility · but high run-time cost and errors are detected late

Why this matters for recursion: recursion is only possible because locals are **stack-dynamic** — each call gets a fresh set. If all variables were statically allocated, every recursive call would overwrite the previous one's data. That single sentence connects the lecturer's Q2 and Q3.

2 Static vs dynamic scoping, and the other parameter-passing models

STATIC (LEXICAL) VS DYNAMIC SCOPING THE NATURAL FOLLOW-UP

Module 4 distinguishes **global, local and block** scope. A deeper question asks **how a non-local name is resolved** — and there are two answers.

BASIS	STATIC (LEXICAL) SCOPING	DYNAMIC SCOPING
Rule	A name refers to the declaration in the **nearest enclosing block in the program text**	A name refers to the declaration in the **most recent active call** at run time
Resolved	At **compile time**, by reading the source	At **run time**, by searching the call chain
Readability	**High** — you can tell what a name means by reading	**Low** — the meaning depends on who called
Reliability	High	Low — a caller can accidentally capture a name
Cost	Cheap — addresses known in advance	Expensive — a run-time search
Used by	**Almost all modern languages** — C++, Python, JavaScript, Java	Early LISP, some shell languages, Perl's `local`

```
x = 10                      # global

def show():
    print(x)                # which x?

def caller():
    x = 99                  # local to caller
    show()

caller()

STATIC scoping  → prints 10   ← Python's answer
DYNAMIC scoping → prints 99   (would look up the caller)
```

The conclusion to write: languages overwhelmingly chose **static scoping** because it lets a reader — and a compiler — determine the meaning of every name **from the program text alone**, which serves the design goals of **readability and reliability** from Module 1.

REFERENCING ENVIRONMENT

The **referencing environment** of a statement is **the complete collection of names visible at that point**. Under static scoping it is the local declarations plus those of all enclosing scopes; under dynamic scoping it is the locals of every active call. It is the formal way of saying "what is in scope here".

THE FIVE PARAMETER-PASSING MODELS EXTENDS TEST Q4

Module 4 gives **by value** and **by reference**. The full family, classified by **direction of data flow**, is:

MODEL	MODE	HOW IT WORKS
Pass by value	**in**	The value is **copied into** the parameter; the caller's variable is never touched
Pass by result	**out**	Nothing is passed in; the parameter's **final value is copied back** to the caller on return
Pass by value-result	**in out**	Copied in **and** copied back on return — also called **copy-restore**
Pass by reference	**in out**	The **address** is passed, so the function works directly on the caller's variable — **no copying**
Pass by name	—	The argument is **textually substituted** for the parameter and re-evaluated on every use — powerful, but confusing and now rare

> **Value-result vs reference — the classic exam distinction.** Both let a function change the caller's variable, but **value-result works on a copy and writes back at the end**, while **reference works on the original throughout**. They differ if the same variable is passed twice, or if the function fails part-way: reference leaves partial changes behind, value-result does not.

DESIGN CONSIDERATIONS FOR PARAMETER PASSING

1. **Efficiency** — copying large objects is expensive, which argues for reference
2. **Safety** — one-way (in mode) passing protects the caller's data
3. **Whether results should be returned through parameters** at all, or only through return values

HOW THE THREE COURSE LANGUAGES ACTUALLY DO IT

LANGUAGE	MECHANISM
C++	**By value by default**; **by reference** with &; **by pointer** with * and &. The programmer chooses explicitly — hence "parameter passing complexity: **high**"

LANGUAGE	MECHANISM
Python	**Pass-by-object-reference** — mutating the object affects the caller, rebinding the name does not
JavaScript	Primitives **by value**; objects and arrays **by reference to the object** — the same split as Python

3 Programming paradigms

Module 4 contrasts **imperative** and **functional** control flow. That is one cut through a larger classification, and "compare programming paradigms" is a standard question on this course.

PARADIGM	CORE IDEA	LANGUAGES	STRENGTHS AND WEAKNESSES
Imperative / procedural	Describes **how** a task is performed, as a sequence of **statements that change program state**. Built on the **von Neumann architecture** — variables model memory cells, assignment models storing	C, C++, Pascal, Python, JavaScript	Efficient and close to the machine · but state changes make programs harder to reason about
Object-oriented	Programs are **objects that combine data and behaviour**, using encapsulation, inheritance and polymorphism	C++, Java, Python, JavaScript	Strong modelling of real-world entities, reuse through inheritance · but added complexity and overhead
Functional	Describes **what** is to be computed by **applying functions**, avoiding changeable state and **side effects**	Haskell, Lisp, ML; supported in Python and JavaScript	Simpler code, easier reasoning, easier parallelism · but unfamiliar and sometimes less efficient
Logic / declarative	Programs are **facts and rules**; the system **infers** answers rather than following steps	**Prolog** — the 4GL of Module 1	Excellent for AI, reasoning and knowledge representation · but limited control over efficiency
Scripting	**Interpreted**, high-level languages for automating tasks and gluing components together	Python, JavaScript, Bash	Rapid development · but slower execution and late error detection

The von Neumann connection — a strong opening line. Imperative languages dominate because they mirror the **von Neumann architecture**, in which instructions and data share one memory and are processed sequentially. **Variables correspond to memory cells, assignment to storing a value, and iteration to the efficient repetition of instructions already in memory.** This is also why **iteration is faster than recursion** on such machines — a point that ties straight back to the lecturer's Q2.

Multi-paradigm languages. The three course languages are all multi-paradigm: **C++** supports procedural, object-oriented and generic programming; **Python** supports procedural, object-oriented and functional; **JavaScript** supports procedural, object-oriented (prototype-based) and functional. A question asking which paradigm a language "is" is usually asking you to notice exactly that.

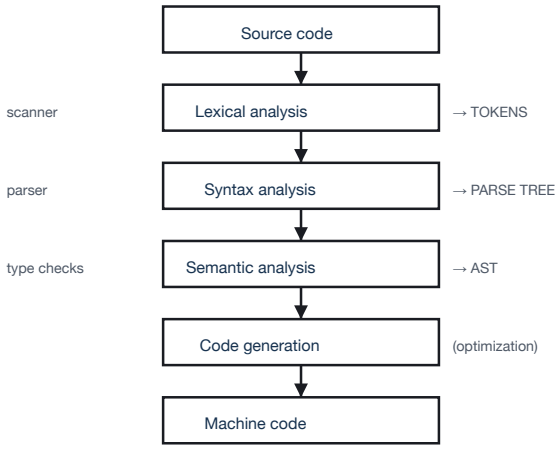
4 Translation — compilation, interpretation and the phases

THE THREE IMPLEMENTATION METHODS

METHOD	HOW IT WORKS	EXAMPLE
Compilation	The **whole program is translated to machine code before execution**; the result runs directly on hardware	C++
Pure interpretation	The source is **translated and executed statement by statement** by an interpreter, with no machine-code output	Early BASIC; Python conceptually
Hybrid	The source is compiled to an **intermediate bytecode**, which is then interpreted or **JIT-compiled** at run time	Java, **JavaScript**, Python's `.pyc`

BASIS	COMPILED	INTERPRETED
Translation	Whole program, **before** running	Line by line, **during** running
Execution speed	**Fast**	Slower
Error detection	All syntax errors found **before** execution	Found only when the line is **reached**
Portability	Machine-specific binary	**Portable** source
Development speed	Slower — recompile to test	**Faster** — run immediately
Memory	Needs no translator at run time	**Interpreter must be present**

THE PHASES OF COMPILATION — AND WHERE MODULE 3'S TREES LIVE



Module 3's parse trees and ASTs are the outputs of phases 2 and 3.

PHASE	WHAT IT DOES
1. Lexical analysis	Groups characters into **tokens** — keywords, identifiers, literals, operators. Catches illegal characters
2. Syntax analysis	Checks tokens against the **grammar (BNF)** and builds the **parse tree**. Catches missing semicolons and mismatched brackets
3. Semantic analysis	Checks **meaning** — type compatibility, undeclared variables — and produces the **AST**. This is where **static semantics** is enforced
4. Optimization	Improves the intermediate code without changing its meaning
5. Code generation	Emits **machine or object code** from the AST

The symbol table runs alongside every phase, holding each identifier's **name, type, scope and address** — the six attributes of §1 in one data structure.

5 Grammars and semantics in more depth

BNF AND EBNF

```
BNF
<assign>  ::= <id> = <expr>
<expr>    ::= <expr> + <term> | <term>
<term>    ::= <term> * <factor> | <factor>
<factor>  ::= <id> | ( <expr> )

EBNF adds shorthand
  { }  zero or more repetitions
  [ ]  optional
  ( | ) choice
<expr> ::= <term> { (+ | -) <term> }
```

TERM	MEANING
Terminal	A symbol that appears in the program itself — `+`, `=`, `if`
Non-terminal	A named construct defined by a rule — `<expr>`
Start symbol	The non-terminal a whole program derives from
Derivation	The sequence of rule applications producing a sentence
Ambiguous grammar	One that allows **two different parse trees** for the same statement — a defect, because meaning becomes undefined

How a grammar encodes precedence: in the rules above, `<term>` handles `*` and sits **below** `<expr>`, which handles `+`. Because multiplication is generated deeper in the tree, it is **evaluated first** — exactly what Module 3's parse tree for `a + b * c` shows. That connection is worth a mark on its own.

STATIC AND DYNAMIC SEMANTICS

KIND	WHAT IT DESCRIBES
Static semantics	Rules that can be **checked before execution** but are not expressible in BNF — e.g. "a variable must be declared before use", "operand types must be compatible". Formalised with **attribute grammars**
Dynamic semantics	The **meaning of statements when executed**

THREE WAYS OF DESCRIBING DYNAMIC SEMANTICS

METHOD	IDEA	USED FOR
Operational	Describe meaning by the **changes of state** a statement causes on an abstract machine	Teaching and language manuals
Denotational	Map each construct onto a **mathematical object** (a function) that denotes its meaning	Rigorous specification; hardest to read
Axiomatic	Describe meaning by **logical assertions** — preconditions and postconditions	**Program verification** and correctness proofs

> **The one-line summary if asked: syntax** is described by **BNF**, **static semantics** by **attribute grammars**, and **dynamic semantics** by **operational, denotational or axiomatic** methods. That sentence covers a whole question on language description.

TYPE CHECKING, COERCION AND EQUIVALENCE

- **Type checking** — ensuring operands of an operator are of **compatible types**.
- **Coercion** — an **implicit, automatic type conversion** performed by the language. JavaScript's `"5" + 2 → "52"` is coercion, and it is why weak typing lowers reliability.
- **Casting** — an **explicit** conversion requested by the programmer, e.g. `(double)x`.
- **Name type equivalence** — two variables have the same type only if declared with the **same type name**. Strict, safe.
- **Structural type equivalence** — same type if their **structures are identical**. Flexible, but two unrelated records that happen to match become interchangeable.

6 Language evaluation criteria, in the full form

Module 1 lists five design goals. The standard PPL treatment breaks the first three into **the characteristics that cause them**, which is what turns a five-line answer into a full-page one.

CRITERION	CONTRIBUTING CHARACTERISTIC	WHAT IT MEANS
Readability the ease with which programs can be understood	**Simplicity**	Few constructs, little feature multiplicity, minimal operator overloading. Too many ways to do one thing hurts the reader
	Orthogonality	A **small set of primitives combinable in a small number of ways**, with **few exceptions**. High orthogonality means rules compose predictably
	Data types	Adequate types and structures — a `bool` reads better than an `int` holding 0 or 1
	Syntax design	Meaningful keywords, clear form for compound statements, identifier rules
	Control statements	Well-designed control structures — the argument against `GOTO`
Writability the ease with which programs can be created	**Simplicity and orthogonality**	Fewer constructs to remember, combined consistently
	Expressivity	Convenient ways to specify computations — `total = sum(numbers)` over a written-out loop
	Support for abstraction	The ability to define and use complex structures and operations while ignoring detail — **Module 2's data abstraction**
Reliability performing to specification under all conditions	**Type checking**	Testing for type errors, ideally at **compile time**
	Exception handling	Intercepting run-time errors and taking corrective action rather than crashing
	Restricted aliasing	Limiting the number of names bound to one address — see §1
	Readability and writability	Both feed reliability: code that is hard to read or write is more likely to be wrong
Cost	**Training, writing, compiling, executing, maintaining**	The total cost of ownership — **maintenance often dominates**, which is why Module 1 lists maintainability as a goal in its own right
	Implementation system cost	The cost and availability of compilers and tools, and the language's reliability record

EXCEPTION HANDLING — WORTH KNOWING IN OUTLINE

An **exception** is an **unusual event, erroneous or not, detectable by hardware or software, that requires special processing**. **Exception handling** lets a program intercept it and respond rather than terminate.

C++
```
try {
    risky();
} catch (exception &e) {
    cout << "handled";
}
```

Python
```
try:
    risky()
except Exception:
    print("handled")
```

Its contribution: exception handling improves **reliability**, and Module 1 names it explicitly as one of the ways a language achieves that goal.

THE TRADE-OFFS TO QUOTE IN ANY DESIGN QUESTION

TRADE-OFF	THE TENSION
Simplicity vs expressiveness	Easy to learn and read, versus powerful and concise
Efficiency vs safety	C++'s performance and explicit control, versus Python's productivity and protection
Flexibility vs reliability	Dynamic typing's rapid development, versus static typing's early error detection
Readability vs writability	Terse constructs are quick to write and slow to read — the two goals genuinely conflict
Time vs space	Module 3's data-structure principle: faster usually means more memory

The universal closing sentence for this course. No language optimises every criterion; **language design is the deliberate selection of a point in these trade-offs to suit an intended domain** — which is exactly why Python, JavaScript and C++ look so different while computing the same things.